

Phase 6

Phase 6A – Amazon EKS Ingress Fundamentals & Architecture

Objective

The objective of this phase is to expose applications running inside Amazon EKS using Kubernetes Ingress and AWS Application Load Balancer (ALB).

Prior to this phase, applications were accessible using:

- ClusterIP
- NodePort
- LoadBalancer

While these methods work, they are not ideal for production environments where multiple applications must be exposed through a single entry point.

Ingress provides a centralized mechanism for routing traffic to multiple applications using a single AWS Application Load Balancer.

Learning Objectives

By completing this phase, you will understand:

- What Ingress is
- What an Ingress Controller is
- Why AWS Load Balancer Controller is required
- Difference between ClusterIP, NodePort, LoadBalancer and Ingress
- How ALB integrates with EKS
- Why OIDC and IRSA are required
- End-to-end traffic flow from browser to pod

Deployment

Validation:

```
kubectl get deployments
```

Purpose:

Deployment manages pod lifecycle and ensures desired replica count.

Pods

Validation:

```
kubectl get pods -o wide
```

Expected Output:

```
nginx-deployment-xxxxx
```

Purpose:

Pods contain running containers.

Example:

Pod IP:
172.31.21.188

Existing Services

Validation:

```
kubectl get svc
```

Expected Output:

```
nginx-service  
nginx-nodeport  
nginx-loadbalancer
```

Understanding Service Types

Before learning Ingress, it is important to understand the services already created.

ClusterIP

Service:

```
nginx-service
```

Purpose:

Internal communication only.

Accessible only from inside the cluster.

Traffic Flow:

Pod



ClusterIP Service



Pod

External users cannot access this service.

NodePort

Service:

nginx-nodeport

Purpose:

Expose application through worker node IP and port.

Traffic Flow:

Browser



Worker Node Public/Private IP



NodePort



Service



Pod

Example:

172.31.x.x:30080

LoadBalancer

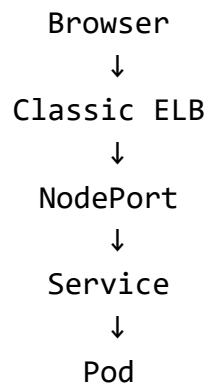
Service:

nginx-loadbalancer

Purpose:

Automatically provisions AWS Classic Load Balancer.

Traffic Flow:



Validation:

kubectl get svc

Expected:

EXTERNAL-IP:

xxxxxxx.ap-south-1.elb.amazonaws.com

Why Ingress Is Needed

Imagine the cluster hosts:

Application A

Application B

Application C

Using Load Balancer Services:

Application A -> ELB 1
Application B -> ELB 2
Application C -> ELB 3

Result:

3 Load Balancers
3 Public Endpoints
Higher Cost
More Management

Ingress solves this problem.

What Is Ingress?

Ingress is a Kubernetes object.

Example:

```
kind: Ingress
```

Ingress contains routing rules.

Example:

```
/          -> nginx-service  
/app1      -> app1-service  
/app2      -> app2-service
```

Important:

Ingress itself does not process traffic.

Ingress only stores rules.

What Is An Ingress Controller?

Ingress Controller is the software component responsible for implementing Ingress rules.

Without an Ingress Controller:

Ingress Exists
Nothing Happens

With an Ingress Controller:

Ingress Created
↓
Controller Reads Rules
↓
Infrastructure Created
↓
Traffic Routed

AWS Load Balancer Controller

For Amazon EKS, the Ingress Controller used is:

AWS Load Balancer Controller

This controller runs as pods inside:

kube-system

Purpose:

- Creates ALBs
- Creates Target Groups
- Creates Listener Rules
- Registers Targets
- Updates ALBs automatically

Why AWS Load Balancer Controller Needs IAM Permissions

The controller creates AWS resources.

Examples:

ALB , Target Groups
Listeners, Listener Rules, Security Groups

Normal Kubernetes pods cannot create AWS resources.

Therefore:

Pod



Service Account



IAM Role



IAM Policy



AWS API Access

must be configured.

This process is called:

IRSA

IAM Roles for Service Accounts.

High-Level Architecture After Phase 6

Final Architecture:

Browser



Application Load Balancer (ALB)



Listener



Target Group



Worker Node



NodePort Service



kube-proxy



nginx Pod

Why ALB Was Chosen

Advantages:

- Layer 7 Load Balancing
- Path-Based Routing
- Host-Based Routing
- SSL Termination
- Single Entry Point
- Multiple Applications Behind One ALB

Example:

/app1 -> Service A

/app2 -> Service B

/app3 -> Service C

One ALB serves multiple applications.

Key Terms Learned In This Phase

Ingress

A Kubernetes object containing routing rules.

Ingress Controller

Component that implements Ingress rules.

AWS Load Balancer Controller

AWS-specific Ingress Controller used in EKS.

ALB

Application Load Balancer created by AWS Load Balancer Controller.

IRSA

IAM Roles for Service Accounts.

Allows Kubernetes Pods to access AWS APIs securely.

OIDC

OpenID Connect.

Provides trust relationship between Kubernetes identities and AWS IAM.

Target Group

Collection of targets registered behind an ALB.

Targets may be:

- EC2 Instances
- NodePorts
- Pod IPs

End of Phase 6A

At this point we understand:

- Why Ingress exists
- Why AWS Load Balancer Controller is required
- Why IAM permissions are required
- How traffic will eventually flow from browser to pod

Phase 6B – OIDC, IRSA, IAM Policy, IAM Role & Service Account Configuration

Objective

The objective of this phase is to grant AWS permissions to the AWS Load Balancer Controller running inside Amazon EKS.

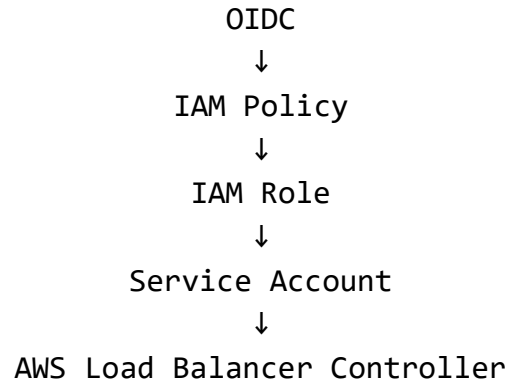
By default:

Pods cannot create AWS resources.

The AWS Load Balancer Controller needs permission to create:

- Application Load Balancers (ALB)
- Target Groups
- Listeners
- Listener Rules
- Security Group Updates

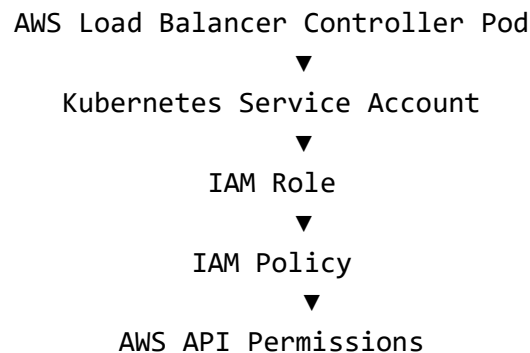
To achieve this securely we configure:



This pattern is called: IRSA

IAM Roles for Service Accounts.

Architecture



Step 1 – Verify Cluster OIDC Issuer

Purpose: Before creating IAM roles for Kubernetes workloads, verify whether the cluster has an OIDC issuer.

Command:

```
aws eks describe-cluster `
  --name mani-eks-cluster `
  --query "cluster.[name,version,identity.oidc.issuer]" `
  --output table
```

Expected Output:

mani-eks-cluster

1.35

<https://oidc.eks.ap-south-1.amazonaws.com/id/xxxxxxx>

Explanation:

The OIDC issuer is the identity provider used by Kubernetes service accounts.

AWS IAM will trust this provider.

Without it:

Service Account

↓

IAM Role

✗ Impossible

Step 2 – Verify Existing OIDC Providers

Command:

```
aws iam list-open-id-connect-providers
```

Lab Output:

```
{
  "OpenIDConnectProviderList": []
}
```

Observation:

No OIDC provider existed.

Therefore one had to be created.

Step 3 – Install eksctl

Purpose:

eksctl simplifies EKS operations.

It is used to:

- Associate OIDC providers
- Create IAM Service Accounts
- Manage EKS integrations

Verify Installation:

```
eksctl version
```

Lab Output:

```
0.227.0
```

Step 4 – Associate OIDC Provider

Command:

```
eksctl utils associate-iam-oidc-provider `
--cluster mani-eks-cluster `
--region ap-south-1 `
--approve
```

Lab Output:

```
IAM Open ID Connect provider is already associated
```

Explanation:

In our environment the provider already existed by the time we ran this command.

This command is safe to run repeatedly.

Step 5 – Validate OIDC Provider

Command: `aws iam list-open-id-connect-providers`

Lab Output:

```
arn:aws:iam::447633182123:oidc-provider/oidc.eks.ap-south-
1.amazonaws.com/id/3CC36F32A671BC8BB549EB65D28F4998
```

Validation Success:

OIDC Provider Present

Why OIDC Is Required

Without OIDC:

Pod



AWS API  Access Denied

With OIDC:

Pod



Service Account



IAM Role



AWS API  Authorized

Step 6 – Download AWS Load Balancer Controller IAM Policy

Purpose:

AWS provides a recommended policy for the AWS Load Balancer Controller.

Download from official GitHub Location:

https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json

Save as: iam-policy.json

Validation: `dir iam-policy.json`

Lab Output: iam-policy.json

Step 7 – Create IAM Policy

Command:

```
aws iam create-policy `
  --policy-name AWSLoadBalancerControllerIAMPolicy `
  --policy-document file:///iam-policy.json
```

Lab Output:

PolicyName:

AWSLoadBalancerControllerIAMPolicy

Policy ARN:

arn:aws:iam::447633182123:policy/AWSLoadBalancerControllerIAMPolicy

Purpose:

Grants permissions required for:

Create ALB

Create Target Group

Modify Listener

Register Targets

Describe Resources

Step 8 – Obtain OIDC Issuer

Command:

```
aws eks describe-cluster `
  --name mani-eks-cluster `
  --query "cluster.identity.oidc.issuer" `
  --output text
```

Lab Output: <https://oidc.eks.ap-south-1.amazonaws.com/id/3CC36F32A671BC8BB549EB65D28F4998>

This value is required for the trust policy.

Step 9 – Create Trust Policy

Purpose:

Defines which Kubernetes Service Account can assume the IAM Role.

Create:

alb-trust-policy.json

Replace:

ACCOUNT_ID

OIDC_ID

with actual values.

Example Structure:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::ACCOUNT_ID:oidc-provider/OIDC_PROVIDER"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "OIDC_PROVIDER:aud": "sts.amazonaws.com",
          "OIDC_PROVIDER:sub": "system:serviceaccount:kube-system:aws-load-balancer-controller"
        }
      }
    }
  ]
}
```

Important:

Only this service account is trusted:

kube-system/aws-load-balancer-controller

Step 10 – Create IAM Role

Command:

```
aws iam create-role `
  --role-name AmazonEKSLoadBalancerControllerRole `
  --assume-role-policy-document file:///alb-trust-policy.json
```

Lab Output:

AmazonEKSLoadBalancerControllerRole

Role ARN:

arn:aws:iam::447633182123:role/AmazonEKSLoadBalancerControllerRole

Purpose:

Identity assumed by the AWS Load Balancer Controller.

Step 11 – Attach IAM Policy To Role

Command:

```
aws iam attach-role-policy `
  --role-name AmazonEKSLoadBalancerControllerRole `
  --policy-arn
arn:aws:iam::447633182123:policy/AWSLoadBalancerControllerIAMPolicy
```

Purpose: Attach permissions to the role.

Without this:

Role Exists

Permissions Do Not

Step 12 – Create IAM Service Account

Initial Attempt

Command:

```
eksctl create iamserviceaccount `
  --cluster=mani-eks-cluster `
  --namespace=kube-system `
  --name=aws-load-balancer-controller `
  --role-name AmazonEKSLoadBalancerControllerRole `
  --attach-role-arn
arn:aws:iam::447633182123:role/AmazonEKSLoadBalancerControllerRole
```

Error:

```
cannot provide --role-name or --role-only
when --attach-role-arn is configured
```

Lesson:

When using:

```
--attach-role-arn
```

do not specify:

```
--role-name
```

Correct Command

```
eksctl create iamserviceaccount `
  --cluster=mani-eks-cluster `
  --namespace=kube-system `
  --name=aws-load-balancer-controller `
  --attach-role-arn
arn:aws:iam::447633182123:role/AmazonEKSLoadBalancerControllerRole `
  --approve `
  --region ap-south-1
```

Lab Output:

created serviceaccount

Step 13 – Validate Service Account

Command:

```
kubectl get sa -n kube-system
```

Expected:

```
aws-load-balancer-controller
```

Step 14 – Verify Service Account Annotation

Command:

```
kubectl describe sa aws-load-balancer-controller -n kube-system
```

Lab Output:

Annotations:

```
eks.amazonaws.com/role-arn:
```

```
arn:aws:iam::447633182123:role/AmazonEKSLoadBalancerControllerRole
```

This annotation is the most important validation.

It proves:

Service Account

|



IAM Role

Connected Successfully

Phase 6B Validation Checklist

OIDC Provider Exists 

IAM Policy Created 

IAM Role Created 

IAM Policy Attached 

Service Account Created 

Service Account Annotated 

IRSA Configured 

End of Phase 6B

At the end of this phase:

AWS Load Balancer Controller

Can now authenticate to AWS
using IAM Roles for Service Accounts (IRSA).

However, the controller software itself is not yet installed.

That is covered in:

Phase 6C – AWS Load Balancer Controller Installation & Verification

Objective

The objective of this phase is to install the AWS Load Balancer Controller into the EKS cluster.

At the end of this phase:

Ingress

|



AWS Load Balancer Controller



AWS APIs

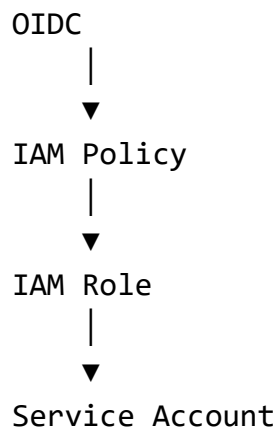
will be fully functional.

The controller will be capable of:

- Creating ALBs
- Creating Target Groups
- Creating Listeners
- Creating Listener Rules
- Managing Ingress Resources

Architecture Before Installation

At the end of Phase 6B we completed:



However:

AWS Load Balancer Controller

✗ Not Installed

At this stage:

Ingress ✗ Will Not Work

because no controller exists to process ingress resources.

Step 1 – Verify Existing System Pods

Command:

```
kubectl get pods -n kube-system
```

Lab Output:

```
aws-node  
aws-node
```

```
coredns  
coredns
```

```
kube-proxy  
kube-proxy
```

Observation:

No AWS Load Balancer Controller pods existed.

This confirmed installation was still required.

Step 2 – Install Required CRDs

What Are CRDs?

CRD = Custom Resource Definition

Purpose: Allows Kubernetes to understand custom AWS objects.

Examples:

```
TargetGroupBinding  
IngressClassParams
```

Without CRDs:

AWS Load Balancer Controller

✗ Cannot Function

Install CRDs

Command: `kubectl apply -k "github.com/aws/eks-charts/stable/aws-load-balancer-controller/crds?ref=master"`

Lab Output:

`albtargetcontrolconfigs.elbv2.k8s.aws created`

`globalaccelerators.aga.k8s.aws created`

`ingressclassparams.elbv2.k8s.aws created`

`targetgroupbindings.elbv2.k8s.aws created`

Validation:

`kubectl get crd`

Expected: `targetgroupbindings.elbv2.k8s.aws` must exist.

Step 3 – Verify Helm

Purpose:

AWS Load Balancer Controller is installed via Helm.

Verify:

`helm version`

Lab Output:

Version:

`v4.2.0`

Validation:

Helm Installed

Step 4 – Add AWS EKS Helm Repository

Command:

```
helm repo add eks https://aws.github.io/eks-charts
```

Lab Output:

"eks" has been added

Step 5 – Update Helm Repository

Command:

```
helm repo update
```

Lab Output:

Successfully got an update from the "eks" chart repository

Purpose:

Downloads latest chart metadata.

Step 6 – Install AWS Load Balancer Controller

Initial Installation

Command Used:

```
helm install aws-load-balancer-controller eks/aws-load-balancer-  
controller `
  -n kube-system `
  --set clusterName=mani-eks-cluster `
  --set serviceAccount.create=false `
  --set serviceAccount.name=aws-load-balancer-controller `
  --set region=ap-south-1 `
  --set vpcId=vpc-xxxxxxx
```

Explanation:

Parameter Breakdown:

`clusterName`

`mani-eks-cluster`

Target EKS cluster.

`serviceAccount.create=false`

Reason:

Service Account already created in Phase 6B.

Helm should not create another one.

`serviceAccount.name`

`aws-load-balancer-controller`

Use existing IRSA-enabled Service Account.

`region`

`ap-south-1`

AWS Region.

`vpcId`

Purpose:

Controller must know which VPC hosts the cluster.

Installation Output

Expected:

STATUS: deployed

Lab Output:

STATUS: deployed

Validation:

Chart Installed Successfully

Step 7 – Verify Controller Pods

Command:

```
kubectl get pods -n kube-system
```

Lab Output:

```
aws-load-balancer-controller-xxxxx
```

```
aws-load-balancer-controller-xxxxx
```

Status:

```
1/1 Running
```

Validation: Controller Running

Step 8 – Verify Controller Logs

Purpose:

Confirm controller can communicate with AWS.

Command:

```
kubectl logs -n kube-system deployment/aws-load-balancer-controller --  
tail=20
```

Lab Output:

```
Starting Controller
```

```
Starting workers
```

```
Starting EventSource
```

Important Observation:

No errors such as:

AccessDenied

UnauthorizedOperation

WebIdentityErr

Validation:

IRSA Working Correctly

Step 9 – Verify IngressClass

Command:

```
kubectl get ingressclass
```

Lab Output:

NAME	CONTROLLER
------	------------

alb	ingress.k8s.aws/alb
-----	---------------------

Meaning:

AWS Load Balancer Controller successfully registered:

Ingress Class:

alb

Future ingress resources will use:

ingressClassName: alb

Step 10 – Verify Helm Configuration

After troubleshooting later in the lab we discovered an issue.

Check current values:

```
helm get values aws-load-balancer-controller -n kube-system
```

Lab Output:

```
clusterName: mani-eks-cluster
```

```
region: ap-south-1
```

```
vpcId: vpc-xxxxxxx
```

Problem:

Incorrect VPC ID was configured.

Actual VPC:

```
vpc-011568294a640b9ed
```

At this stage the controller still appeared healthy.

However:

ALB Provisioning  Failed Later

because controller was referencing the wrong VPC.

This issue was discovered during ingress troubleshooting.

Step 11 – Correct VPC Configuration

Upgrade Helm Release:

```
helm upgrade aws-load-balancer-controller eks/aws-load-balancer-  
controller `
```

```
-n kube-system `
```

```
--set clusterName=mani-eks-cluster `
```

```
--set serviceAccount.create=false `
```

```
--set serviceAccount.name=aws-load-balancer-controller `
```

```
--set region=ap-south-1 `
```

```
--set vpcId=vpc-011568294a640b9ed
```

Expected Output:

STATUS: deployed

REVISION: 2

Step 12 – Validate Updated Configuration

Command:

```
helm get values aws-load-balancer-controller -n kube-system
```

Expected:

vpcId: vpc-011568294a640b9ed

Validation:

Correct VPC Configured

Why The VPC Issue Was Difficult To Detect

Controller Pods:

Running

Logs:

Healthy

IngressClass:

Created

Everything looked correct.

The issue only appeared later when:

Controller Tried To Create ALB

This is a common production troubleshooting scenario.

Phase 6C Validation Checklist

CRDs Installed 

Helm Installed 

Repository Added 

Controller Installed 

Controller Pods Running 

Logs Healthy 

IngressClass Present 

VPC Configuration Corrected 

End of Phase 6C

At the end of this phase:

AWS Load Balancer Controller

Running Successfully

and capable of processing:

Ingress Resources

However: No Ingress Exists Yet

The controller is waiting for ingress resources to be created.

This is covered in

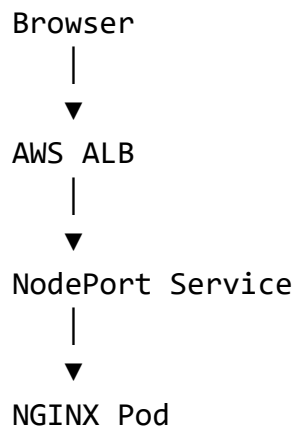
Phase 6D – Ingress Deployment, ALB Provisioning, Troubleshooting & Traffic Flow

Objective

The objective of this phase is to:

- Create an Ingress Resource
- Provision an AWS Application Load Balancer (ALB)
- Route traffic to the NGINX application
- Troubleshoot real-world ingress deployment failures
- Validate end-to-end connectivity

At the end of this phase:



will be fully operational.

Step 1 – Verify Existing Services

Command:

```
kubectl get svc
```

Lab Output:

NAME	TYPE
nginx-service	ClusterIP
nginx-nodeport	NodePort

nginx-loadbalancer LoadBalancer

Purpose:

Confirm application services exist before configuring ingress.

Step 2 – Verify No Existing Ingress

Command:

```
kubectl get ingress
```

Lab Output:

No resources found

Expected.

No ingress had been created yet.

Step 3 – Create Ingress Resource

Create:

```
ingress.yaml
```

Initial Version:

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
```

```
metadata:
```

```
  name: nginx-ingress
```

```
  annotations:
```

```
    alb.ingress.kubernetes.io/scheme: internet-facing
```

```
spec:
```

```
  ingressClassName: alb
```

```
  rules:
```

```
    - http:
```

```
paths:
- path: /
  pathType: Prefix

backend:
  service:
    name: nginx-service
    port:
      number: 80
```

Explanation Of Each Field

`ingressClassName`

`ingressClassName: alb`

Purpose:

Assign ingress ownership to:

AWS Load Balancer Controller

Without this: Controller ignores ingress

`internet-facing`

`alb.ingress.kubernetes.io/scheme: internet-facing`

Purpose:

Create:

Public ALB

instead of:

Internal ALB

`Backend Service`

Initially:


```
service:  
  name: nginx-service
```

which pointed to:
ClusterIP Service

This later became a problem.

Step 4 – Create Ingress

Command:

```
kubectl apply -f ingress.yaml
```

Validation:

```
kubectl get ingress
```

Expected:

NAME

nginx-ingress

Initially:

ADDRESS

empty

This is normal as ALB provisioning takes time.

Step 5 – First Failure

Validation:

```
kubectl describe ingress nginx-ingress
```

Error:

Failed build model

couldn't auto-discover subnets

This was the first major failure.

Root Cause Analysis

Controller attempted:

Ingress



Discover Subnets



Create ALB

Discovery failed.

Step 6 – Verify Cluster Subnets

Command:

```
aws eks describe-cluster `
  --name mani-eks-cluster `
  --query "cluster.resourcesVpcConfig.subnetIds" `
  --output table
```

Lab Output:

subnet-03b34759d8b45a74c

subnet-0896012da39f4bdd0

subnet-017665542b35fe5ce

Step 7 – Inspect Subnet Tags

Command:

```
aws ec2 describe-subnets `
  --filters "Name=vpc-id,Values=vpc-011568294a640b9ed" `
  --query "Subnets[*].[SubnetId,AvailabilityZone,Tags]" `
  --output table
```

Lab Output: Tags = None

Problem Identified: No EKS Tags

Step 8 – Add ELB Tags

Command:

```
aws ec2 create-tags `
  --resources subnet-03b34759d8b45a74c `
  subnet-0896012da39f4bdd0 `
  subnet-017665542b35fe5ce `
  --tags Key=kubernetes.io/role/elb,Value=1
```

Purpose:

Mark subnet as eligible for public ALB deployment.

Step 9 – Add Cluster Ownership Tags

Command:

```
aws ec2 create-tags `
  --resources subnet-03b34759d8b45a74c `
  subnet-0896012da39f4bdd0 `
  subnet-017665542b35fe5ce `
  --tags Key=kubernetes.io/cluster/mani-eks-cluster,Value=shared
```

Purpose:

Associate subnet with cluster.

Step 10 – Validate Subnet Tags

Command:

```
aws ec2 describe-subnets `
  --filters "Name=vpc-id,Values=vpc-011568294a640b9ed" `
  --query "Subnets[*].{Subnet:SubnetId,Tags:Tags}" `
  --output json
```

Expected:

kubernetes.io/role/elb = 1

kubernetes.io/cluster/mani-eks-cluster = shared

Validation Success: Subnet Discovery Fixed

Step 11 – Second Failure

Ingress still failed.

Check: `helm get values aws-load-balancer-controller -n kube-system`

Lab Output:

vpcId: vpc-xxxxxxx

Problem:

Incorrect VPC configured.

Actual VPC:

vpc-011568294a640b9ed

Step 12 – Correct VPC Configuration

Command:

```
helm upgrade aws-load-balancer-controller eks/aws-load-balancer-
controller `
  -n kube-system `
  --set clusterName=mani-eks-cluster `
```

```
--set serviceAccount.create=false `
--set serviceAccount.name=aws-load-balancer-controller `
--set region=ap-south-1 `
--set vpcId=vpc-011568294a640b9ed
```

Validation:

```
helm get values aws-load-balancer-controller -n kube-system
```

Expected:

```
vpcId: vpc-011568294a640b9ed
```

Step 13 – Recreate Ingress

Delete:

```
kubect1 delete ingress nginx-ingress
```

Create Again:kubect1 apply -f ingress.yaml

Step 14 – Third Failure

Validation: kubect1 describe ingress nginx-ingress

Error: TargetGroup port is empty When using Instance targets,
service must be NodePort

Root Cause

Ingress pointed to:nginx-service

which was: ClusterIP

AWS ALB Controller default target type: instance

Traffic Flow:

ALB



Worker Node



NodePort

Therefore: NodePort Required

Step 15 – Update Ingress

Modify backend service.

Old:

```
service:
  name: nginx-service
```

New:

```
service:
  name: nginx-nodeport
```

Final ingress.yaml:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```
metadata:
  name: nginx-ingress
```

```
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
```

```
spec:
  ingressClassName: alb
```

```
rules:
- http:
  paths:
  - path: /
    pathType: Prefix

  backend:
    service:
      name: nginx-nodeport
      port:
        number: 80
```

Apply: `kubectl apply -f ingress.yaml`

Step 16 – Validate ALB Creation

Command:

```
kubectl describe ingress nginx-ingress
```

Lab Output:

SuccessfullyReconciled

Validation Success:

ALB Created Successfully

Step 17 – Obtain ALB DNS

Command:

```
kubectl get ingress
```

Lab Output:

ADDRESS

k8s-default-nginxing-78206b0cff-1327154492.ap-south-

1.elb.amazonaws.com

Validation Success:

Public ALB Available

Step 18 – Browser Validation

Open:

<http://k8s-default-nginxing-78206b0cff-1327154492.ap-south-1.elb.amazonaws.com>

Expected: Welcome to nginx!

Lab Result: Success

NGINX page displayed.

Final Traffic Flow

Browser

↓

AWS Application Load Balancer

↓

Listener

↓

Target Group

↓

Worker Node

↓

NodePort

↓

kube-proxy

↓

nginx-nodeport Service

↓

nginx Pod

↓

NGINX Response

↓

Browser

Detailed Backend Flow

When a user opens:

<http://ALB-DNS>

Flow:

1. Browser sends HTTP request.
2. ALB receives request.
3. Listener evaluates routing rules.
4. Listener forwards request to Target Group.
5. Target Group forwards request to worker node.
6. NodePort receives request.
7. kube-proxy forwards traffic.
8. Service selects matching pod.
9. Pod serves NGINX page.
10. Response returns through same path.

Troubleshooting Summary

Issue #1 couldn't auto-discover subnets

Cause: Missing subnet tags

Fix:

kubernetes.io/role/elb=1

kubernetes.io/cluster/mani-eks-cluster=shared

Issue #2 ALB not provisioning correctly

Cause: Incorrect VPC configured

Fix: Helm upgrade with correct VPC ID

Issue #3 TargetGroup port is empty

Cause: Ingress pointed to ClusterIP Service

Fix: Use NodePort Service

Phase 6 Completion Checklist

OIDC ☒

IRSA ☒

IAM Policy ☒

IAM Role ☒

Service Account ☒

CRDs ☒

AWS LB Controller ☒

IngressClass ☒

Ingress ☒

Subnet Discovery ☒

ALB Creation 

Target Group 

Browser Access 

End-To-End Traffic Flow 

Lessons Learned

Ingress is only a routing object.

Ingress Controller performs the actual work.

AWS Load Balancer Controller requires:

- OIDC
- IAM Role
- Service Account
- IAM Policy

Subnet tagging is mandatory for ALB discovery.

NodePort is required when using ALB instance targets. Healthy controller pods do not always guarantee successful ALB creation.

Validation and troubleshooting are critical.